Project Structure, Coding Conventions and Open Questions

Comprehensive Management System for Radiodiagnostics and Medical Imaging (SESCAM) — Software Engineering II, Course Project 2026/2027. Team of 6. Version 0.2, aligned with the architecture proposal v0.3 (A. Aceña, 04/11/2026) and the decisions D-01 to D-13 in `DECISIONS.md`. To be ratified at the team meeting.

1. Purpose and scope

This document has the three things this part of the team was asked for: a Java/Maven project structure, a draft of basic coding conventions, and a list of technical questions and possible blockers.

Everything here is a proposal to discuss. The structure follows the architecture proposal v0.3 and the decisions already written in `DECISIONS.md`; the points that are still open are in section 5. Whatever the team agrees goes into `DECISIONS.md` through a pull request.

Version 0.2 is the definitive version of this document. Compared with the provisional draft, it renames the modules to match the architecture proposal (singular names, and the integration module split into three adapters), moves the REST controllers into the modules that expose them, and fixes the numbering of the components: C1 to C17 are the 17 component modules, and `radio-app` is not a component.

The workflow side (branches, pull requests, commits, CI, configuration test) is in the companion document *Development Workflow and Configuration Plan – SESCAM Radiodiagnostics*. We only point to it: §3 branches, §3.3 pull requests, §3.4 commit format, §3.7 agent rules, §3.8 definition of done, §5 configuration test.

2. Constraints and decided items

Item	What we are working with
Language and build	Java 17 (bytecode target), Maven as a multi-module project: every component is its own submodule. Spring Boot 3.5.16 (D-04).
How modules talk	Only through interfaces. A module never uses another module's implementation classes (D-03).
Style	Modular monolith: 17 modules deployed as one Spring Boot application, `radio-app` (D-01).
Security	Spring Security configured in `radio-auth`; permissions are checked on the server (D-09).
Data	Each module owns its tables and repositories. `radio-metrics` is the only exception, with read-only queries (D-06).
Process	Unified Process, 1 requirement = 1 use case = 1 iteration, SemVer per component, Git Flow (workflow plan §3).
Decided by the team	Similar functionalities go in the same module. English everywhere. OpenCode as the agent tool. The coordinator administers the repository.
Confirmed by the professor	Repository `2026-ISO2-A1.04` can be public. SSO, health records and RIS/PACS are simulated. Fictitious data only.
Tools	Eclipse and Visual Paradigm are the course tools. Agentic environments are also allowed.
Deadlines	Theory report: **13 November 2026**. Practical delivery: **15 December 2026**.

3. Project structure

3.1 Repository layout

```
radio-sescam/                      repository root
├── pom.xml                        parent POM (packaging: pom)
├── mvnw, mvnw.cmd, .mvn/          Maven Wrapper, pinned to 3.9.16
├── AGENTS.md CONTEXT.md PLAN.md DECISIONS.md README.md   changed only via PR
├── .github/workflows/ci.yml       mvn -B verify on every pull request
├── docs/                        structure.md (module map), architecture/
├── radio-common/                  shared value objects and exceptions
├── radio-auth/                    C1 users, roles, permissions (+ ISsoGateway)
├── radio-configuration/           C2 system parameters
├── radio-catalogue/               C3 examination types and guides
├── radio-patient/                 C4 patients (+ IHceGateway)
├── radio-request/                 C5 requests; the only owner of the status
├── radio-scheduling/              C6 slots, bookings, waiting lists, worklists
├── radio-execution/               C7 exam execution and incidents
├── radio-report/                  C8 reports, versions and signature
├── radio-image/                   C9 DICOM viewer access (+ IPacsGateway)
├── radio-feedback/                C10 incidents and surveys
├── radio-metrics/                 C11 one metrics engine for the whole system
├── radio-notification/            C12 reminders, inbox, email/push stub
├── radio-audit/                   C13 append-only audit trail
├── radio-jobs/                    C14 timer: due reminders and follow-ups
├── radio-sso-adapter/             C15 SSO client + simulator
├── radio-hce-adapter/             C16 HCE/admission client + simulator
├── radio-pacs-adapter/            C17 RIS/PACS client + simulator
└── radio-app/                     composition root: boot, wiring, UI pages

Each module:
├── pom.xml                        parent = radio-sescam; no versions here
├── CHANGELOG.md                   SemVer history of this component
└── src/main/java/es/uclm/sescam/<module>/
    ├── api/  domain/  application/  infrastructure/
    └── ...
```

The parent lists only the modules that exist. Today: `radio-common`, `radio-audit`, `radio-catalogue` and `radio-app`. The rest of the tree is the target: a module is created when an iteration needs it, never to fill an empty folder.

3.2 Module list (17 components + `radio-app` + `radio-common`)

Module	What it groups (architecture component)	Block
`radio-common`	Shared value objects and domain exceptions. No component.	0
`radio-audit`	C13. Append-only audit trail; no update or delete.	1
`radio-catalogue`	C3. Examination types and their attributes, clinical guides.	1
`radio-auth`	C1. Login, users, roles, permissions; configures Spring Security.	1
`radio-configuration`	C2. Room schedules, policies, priority levels, reminder lead time.	1
`radio-sso-adapter`	C15. Corporate SSO client and its simulator.	1
`radio-patient`	C4. Patient registration and updates.	2
`radio-request`	C5. Prescriptions, follow-up series, request status (single owner).	2
`radio-notification`	C12. Reminders, preparation, inbox, email/push stub.	2
`radio-scheduling`	C6. Slots, booking, cancellation, waiting lists, worklists.	2
`radio-jobs`	C14. Timer: due reminders and follow-up generation.	2
`radio-hce-adapter`	C16. HCE and admission client (FHIR/HL7) and its simulator.	2
`radio-execution`	C7. How the exam was performed, incidents.	3
`radio-report`	C8. Drafts, review, signature, versions, patient history.	3
`radio-image`	C9. DICOM viewer access and policy.	3
`radio-pacs-adapter`	C17. RIS/PACS client (DICOM) and its simulator.	3
`radio-feedback`	C10. Incidents and satisfaction surveys.	4
`radio-metrics`	C11. One metrics engine: physician, operational, global.	4
`radio-app`	Not a component. Starts the application, wires the modules, serves the UI pages.	0

> **Persistence is not a module.** Each module owns its repositories and its tables in its own `infrastructure` package (D-06). A shared persistence module would give every module access to every table, which is exactly what the architecture forbids. The only exception is `radio-metrics`, which runs read-only queries for the statistics.

D-02 counts 17 component modules plus `radio-app`. We add `radio-common`, a technical module with no component, only for the shared value objects (`UserRef`, `DomainException`): with it the repository has 19 POMs. If the team prefers exactly 18, those few types move to the modules that use them (P-09).

3.3 Packages inside a module

Package	What goes there	Rule
`es.uclm.sescam.<module>.api`	Interfaces, immutable records (commands, summaries) and the REST controllers of the module.	The **only** package importable from another module. We propose to move the controllers to a separate `web` package, so `api` stays a contract (P-10).
`...domain`	Entities, value objects and repository ports.	Never imported from another module.
`...application`	Use-case implementations and transaction boundaries.	Never imported from another module.
`...infrastructure`	JPA repositories, adapters, mappers, framework glue.	Never imported from another module (see 4.7).

Each module exposes its own REST api (`/api/patients`, `/api/requests`, ...) and the UI pages call them over HTTP. `radio-app` additionally has `es.uclm.sescam.app.config` with the wiring and the static pages. No business module has an `app` package.

3.4 What we build in Iteration 0

- Parent POM with the four existing modules, Java 17, UTF-8, the Boot BOM, the enforcer and the wrapper.
- `radio-common`, `radio-audit`, `radio-catalogue` and `radio-app`.
- One vertical slice: HU-23, register and list examination types, with its audit entry and a JUnit 5 test.
- In-memory adapters instead of a database: the database decision is still open (P-04), and an empty schema would be overengineering.
- `ci.yml`, `AGENTS.md`, `CONTEXT.md`, `PLAN.md`, `DECISIONS.md`, `docs/structure.md` and the wiki index.
- The other 15 modules are only planned (3.2). Empty modules are technical debt, not progress.

3.5 Where the contracts live

Each module owns the interfaces of its component, in its own `api` package (D-03). The alternative, one central `radio-api` module with every interface, was discarded: the architecture already assigns each of its 17 interfaces to a module, so the contract belongs there,

and a central package would end up being a dumping ground where any change rebuilds everything.

3.6 Java and Maven compatibility (checked against Maven Central and the Spring docs)

Piece	Version	Why
JDK	17 (bytecode)	`release=17`; the enforcer accepts JDK 17 or newer at runtime.
Maven	3.9.16 (wrapper)	Maven 4 is still a release candidate (4.0.0-rc-7).
Compiler plugin	3.16.0	The 4.0.0 line is still beta.
Surefire	3.6.1	JUnit Platform support.
Enforcer	3.7.0	Requires Maven 3.6.3+.
JUnit	managed by the Boot BOM	Do not pin it by hand: it must match the Boot line.
ArchUnit	1.5.1	`archunit-junit5` is built on JUnit 5; do not mix JUnit 5 and 6.
Spring Boot	3.5.16	Java 17 baseline, JUnit 5, stable starter names. Boot 4.1 also runs on Java 17, but brings JUnit 6 and renamed starters; not worth the risk for six weeks.

4. Coding conventions (draft)

These rules are about **code**. Branches, commits, pull requests and agent rules live in the workflow plan and are only referenced here.

4.1 Naming

- Classes in `PascalCase`, methods in `camelCase`, booleans starting with `is` / `has` / `can`, constants in `UPPER_SNAKE_CASE`.
- No abbreviations (`appt` → `appointment`), no numbers in names (`Service2`), no shorthands (`mName`). Packages lowercase and singular.
- Interfaces keep the `I` prefix (`IPatientService`), as in the architecture proposal and as generated by Visual Paradigm (P-11). Records and value objects do not.
- One class, one job. No wildcard imports, no unused imports, no commented-out code.

4.2 Formatting

- 4 spaces (no tabs), max 120 columns, `K&R` braces, final newline at the end of every file.
- UTF-8 everywhere and LF endings through `.gitattributes`, so Windows, macOS and Linux produce the same diff.
- The Eclipse formatter profile lives in the repository; formatting is not a review topic.

4.3 Comments and documentation

- Every public type in an `api` package needs a Javadoc with one sentence saying what it is for (+ `@param` / `@return` when it is not obvious). Internal code needs no boilerplate.
- Comments say **why**, not what. English everywhere. A `TODO` needs an issue number: `// TODO(#42)`.

4.4 Design

- Modules talk only through `api`. Never import another module's `domain`, `application` or `infrastructure` package.
- Shared data classes are immutable records: no public fields, no setters, no JPA entities in `api`.
- Do not return `null`: use `Optional`. `null` only for a missing optional parameter.
- Domain exceptions are unchecked; never swallow an exception and never `printStackTrace()` — use the logger. No static mutable state, no business logic in DTOs or UI classes.

4.5 Dependencies

- Versions live only in the parent `dependencyManagement`; modules declare `groupId`, `artifactId` and scope.
- No new dependency or tool without a line in `DECISIONS.md` (workflow plan §3.7, rule 1).
- Released versions from Maven Central only. The parent pins our own modules so they cannot drift.

4.6 Tests

- JUnit 5. Unit tests are `<Class>Test`, integration tests `<Class>IT`, in the same package as the class they test.
- One behaviour per test, named after it (`rejectsDuplicatedCodeIgnoringCase`), written Arrange–Act–Assert.
- Fictitious data only. `mvn -B verify` must be green locally and in CI before review.

4.7 Rules that the build enforces

Two cheap rules turn this document into a build failure. **maven-enforcer** in the parent checks the JVM (17+), Maven (3.9.x), the bytecode property (`release=17`) and duplicate dependency versions. An **ArchUnit test in `radio-app`** fails if a module imports another module's `domain`, `application` or `infrastructure`. Checkstyle and SpotBugs can wait; the specification asks us to avoid overengineering.

5. Technical questions and possible blockers

5.1 For the team

Question	Why it matters
Keep `radio-common` as a technical module (19 POMs) or move its two types and stay at 18 (P-09)?	It changes the parent POM and the module list.
REST controllers inside `api` (D-03) or in a separate `web` package (P-10)?	Decides what other modules can import from us.
Keep the `I` prefix on interfaces (P-11)?	Renaming 17 interfaces later touches every file.
Does `radio-catalogue` record the audit entry, or does the API layer do it (P-12)?	The architecture dependency table leaves the catalogue without dependencies.
Does `radio-metrics` query the tables or the query APIs of the other modules (P-13)?	D-06 allows read-only queries; the other option is slower but cleaner.
Which database (P-04) and which UI technology (P-05)?	Needed before Iteration 1, not for Iteration 0.
Does everyone have JDK 17+ and Maven working, or do we standardise on `./mvnw` only?	Entry requirement for the configuration test (workflow plan §5.7, step 10).
Which OpenCode model do we all use, and who writes the shared `AGENTS.md`?	Different rules give different code styles.
Who owns the parent POM, and who is the second repository administrator?	One administrator is a merge bottleneck and a single point of failure.
What coverage do we require, and how do we split the theory report (13 November)?	The Definition of Done needs a measurable number; the report is graded separately.

5.2 For the professor

Question	Context
One tag per module (`radio-scheduling-1.3.0`) or a single global tag?	With per-component versions, `v1.3.0` alone does not say which module it is.
Is 1 use case = 1 iteration strict for Block 1, or can an iteration group a few small use cases?	Taken literally, the 1:1 rule becomes a pseudo-waterfall, as the specification itself warns.
Are PlantUML diagrams imported into Visual Paradigm enough, or is the `.vpp` file required?	The specification asks for both.
Will branch protection, required checks, Actions minutes and the wiki work on the public repository?	The whole workflow depends on them.

5.3 Risks, blockers and how we avoid them

Risk / blocker	Impact	How we deal with it
Two documents that describe different module names or counts.	High	The architecture proposal and this document use the same names (D-13); the module map lives in `docs/structure.md` and every change goes through a pull request.
Starting from a single Maven module “and splitting later”.	High	The specification requires one submodule per component. Multi-module from the first commit; only empty modules are avoided, not POMs.
The repository has 3 branches nobody recognises and no Maven project yet.	High	Coordinator audits the branches before enabling protection; keep only `main` and `develop`; push the skeleton as the one allowed first push (§5.7.1).
Branch protection or required checks are not available on this repository.	High	Test and record it in the wiki during Iteration 0; otherwise the fallback is review policy only, which is weaker evidence.
It builds on my machine but not in CI.	High	Wrapper 3.9.16 + Temurin 17 in `ci.yml`; “clone and run `./mvnw -B verify`” as the team entry checklist.
Modules with their own versions drift apart and break the final integration.	High	The parent pins every inter-module version, and every PR builds all modules, so drift shows up in week one.
The 1 requirement : 1 use case : 1 iteration rule is read literally, and everything lands in one integration at the end.	High	Each iteration delivers a small tested slice; nothing waits for the last iteration.
Only one person can protect branches, merge and release; dates compress towards 13 November and 15 December.	Medium	Second administrator; Iteration 0 is only the skeleton; when a date slips we do a post-mortem and re-plan.

6. What we are not deciding yet

Not decided in this document: the open points P-09 to P-13 of section 5.1, the client technology, the database, the tag strategy, the Java version beyond the provisional 17, coverage and static analysis. All of it is open for the team meeting and `DECISIONS.md`.